Lecture 03:

Gradient-Based Optimization and Hessians

Matthew Wicker

13 October 2026

Ridge regression, step by step

At the end of lecture 02 we added a penalty on the size of the parameters. Let us derive its minimizer slowly, keeping track of every shape.

$$\overbrace{\mathcal{L}_{\text{reg}}(\theta, \lambda)}^{1\times 1} = \overbrace{\Big\| \underbrace{\boldsymbol{X}}_{m\times n} \underbrace{\theta}_{n\times 1} - \underbrace{\boldsymbol{y}}_{m\times 1} \Big\|_2^2}^{1\times 1} + \overbrace{\underbrace{\lambda}_{1\times 1} \Big\| \underbrace{\theta}_{n\times 1} \Big\|_2^2}^{1\times 1}$$

X: *m* data points (rows), *n* features (columns). θ: one parameter per feature. ***y***: one label per data point.

λ ≥ 0 is a single number: a hyper-parameter we choose.

every term is a scalar (1 × 1): a loss is one number.

Step 1: expand the loss

$$\mathcal{L}_{\text{reg}}(\theta, \lambda) = \|\boldsymbol{X}\theta - \boldsymbol{y}\|_2^2 + \lambda\|\theta\|_2^2$$

$$= (\boldsymbol{X}\theta - \boldsymbol{y})^\top(\boldsymbol{X}\theta - \boldsymbol{y}) + \lambda\,\theta^\top\theta$$

$$= \underbrace{\theta^\top \boldsymbol{X}^\top \boldsymbol{X}\theta}_{1\times 1} - 2\underbrace{\boldsymbol{y}^\top \boldsymbol{X}\theta}_{1\times 1} + \underbrace{\boldsymbol{y}^\top \boldsymbol{y}}_{1\times 1} + \lambda\,\underbrace{\theta^\top\theta}_{1\times 1}$$

a squared norm is an inner product: $\|\boldsymbol{v}\|_2^2 = \boldsymbol{v}^\top\boldsymbol{v}$

multiply out as in lecture 02: $\theta^\top\boldsymbol{X}^\top\boldsymbol{y}$ is a scalar, so it equals its transpose $\boldsymbol{y}^\top\boldsymbol{X}\theta$, and the two middle terms combine

four scalars

Step 2: take the gradient

the rules from lecture 02 (numerator layout: the gradient of a scalar is a row vector, $1 \times n$)

$$\nabla_\theta(\theta^\top A\theta) = \theta^\top(A + A^\top), \qquad \nabla_\theta(c^\top\theta) = c^\top, \qquad \nabla_\theta(\theta^\top\theta) = 2\theta^\top, \qquad \nabla_\theta(\text{const}) = 0^\top$$

$$\begin{aligned}
\nabla_\theta \mathcal{L}_{\text{reg}} &= \nabla_\theta(\theta^\top X^\top X\theta) - 2\nabla_\theta(y^\top X\theta) + \nabla_\theta(y^\top y) + \lambda\nabla_\theta(\theta^\top\theta) \\
&= \theta^\top\left(X^\top X + (X^\top X)^\top\right) - 2y^\top X + 0^\top + \lambda 2\theta^\top \\
&= \underbrace{2\theta^\top X^\top X}_{1\times n} - \underbrace{2y^\top X}_{1\times n} + \underbrace{2\lambda\theta^\top}_{1\times n}
\end{aligned}$$

apply one rule to each term; the constant $y^\top y$ has zero gradient

$X^\top X$ is symmetric, $(X^\top X)^\top = X^\top X$, so the first term doubles

every term is a row vector, $1 \times n$: we can add them

Step 3: set the gradient to zero

$$\underbrace{\mathbf{0}^\top}_{1 \times n} = 2\,\theta^\top \boldsymbol{X}^\top \boldsymbol{X} - 2\,\boldsymbol{y}^\top \boldsymbol{X} + 2\lambda\,\theta^\top$$

$$\mathbf{0}^\top = \theta^\top \boldsymbol{X}^\top \boldsymbol{X} - \boldsymbol{y}^\top \boldsymbol{X} + \lambda\,\theta^\top$$

divide by 2

$$\theta^\top \boldsymbol{X}^\top \boldsymbol{X} + \lambda\,\theta^\top = \boldsymbol{y}^\top \boldsymbol{X}$$

move $\boldsymbol{y}^\top \boldsymbol{X}$ to the right

the zero is a row vector of zeros, $1 \times n$, like the gradient

Step 4: factor out $\theta^\top$, carefully

$$\theta^\top \boldsymbol{X}^\top \boldsymbol{X} + \lambda \theta^\top = \boldsymbol{y}^\top \boldsymbol{X}$$

$$\theta^\top \big(\underbrace{\boldsymbol{X}^\top \boldsymbol{X}}_{n \times n} + \underbrace{\lambda}_{1 \times 1} \big) = \boldsymbol{y}^\top \boldsymbol{X} \qquad ?$$

an $n \times n$ matrix plus a scalar is not defined: the shapes do not match

$$\lambda \theta^\top = \lambda \theta^\top \boldsymbol{I} = \theta^\top (\lambda \boldsymbol{I})$$

multiplying by the identity changes nothing: $\theta^\top \boldsymbol{I} = \theta^\top$, so $\lambda\theta^\top = \theta^\top(\lambda \boldsymbol{I})$

$$\theta^\top \boldsymbol{X}^\top \boldsymbol{X} + \theta^\top (\lambda \boldsymbol{I}) = \theta^\top \big(\underbrace{\boldsymbol{X}^\top \boldsymbol{X}}_{n \times n} + \underbrace{\lambda \boldsymbol{I}}_{n \times n} \big) = \boldsymbol{y}^\top \boldsymbol{X}$$

now both terms are $n \times n$ matrices, and $\theta^\top$ factors out

What is λI?

$$\lambda \boldsymbol{I} = \lambda \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \lambda & 0 & 0 \\ 0 & \lambda & 0 \\ 0 & 0 & \lambda \end{bmatrix}$$

$$\theta^\top (\lambda \boldsymbol{I}) = \begin{bmatrix} \theta_1 & \theta_2 & \theta_3 \end{bmatrix} \begin{bmatrix} \lambda & 0 & 0 \\ 0 & \lambda & 0 \\ 0 & 0 & \lambda \end{bmatrix} = \begin{bmatrix} \lambda\theta_1 & \lambda\theta_2 & \lambda\theta_3 \end{bmatrix} = \lambda\, \theta^\top \quad \checkmark$$

$\lambda \boldsymbol{I}$ acts exactly like the number λ on $\theta^\top$, but it is an $n \times n$ matrix

$$\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} + \begin{bmatrix} \lambda & 0 & 0 \\ 0 & \lambda & 0 \\ 0 & 0 & \lambda \end{bmatrix} = \begin{bmatrix} a_{11} + \lambda & a_{12} & a_{13} \\ a_{21} & a_{22} + \lambda & a_{23} \\ a_{31} & a_{32} & a_{33} + \lambda \end{bmatrix}$$

added to $\boldsymbol{X}^\top \boldsymbol{X}$, it adds λ to the diagonal only (here $a_{ij} = (\boldsymbol{X}^\top \boldsymbol{X})_{ij}$)

Step 5: solve for θ

$$\theta^\top (\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I}) = \boldsymbol{y}^\top \boldsymbol{X}$$

$$(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I})^\top \theta = \boldsymbol{X}^\top \boldsymbol{y}$$

$$(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I})\, \theta = \boldsymbol{X}^\top \boldsymbol{y}$$

$$\theta_\lambda = \underbrace{(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I})^{-1}}_{n \times n} \underbrace{\boldsymbol{X}^\top}_{n \times m} \underbrace{\boldsymbol{y}}_{m \times 1}$$

transpose both sides: $(\theta^\top \boldsymbol{A})^\top = \boldsymbol{A}^\top \theta$

$\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I}$ is symmetric

multiply by the inverse (it exists for every $\lambda > 0$)

the shapes chain: $(n \times n)(n \times m)(m \times 1)$ gives $n \times 1$, one value per parameter

The ridge regression estimator

Lecture 02 (no penalty):

$$\theta_{\text{OLS}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

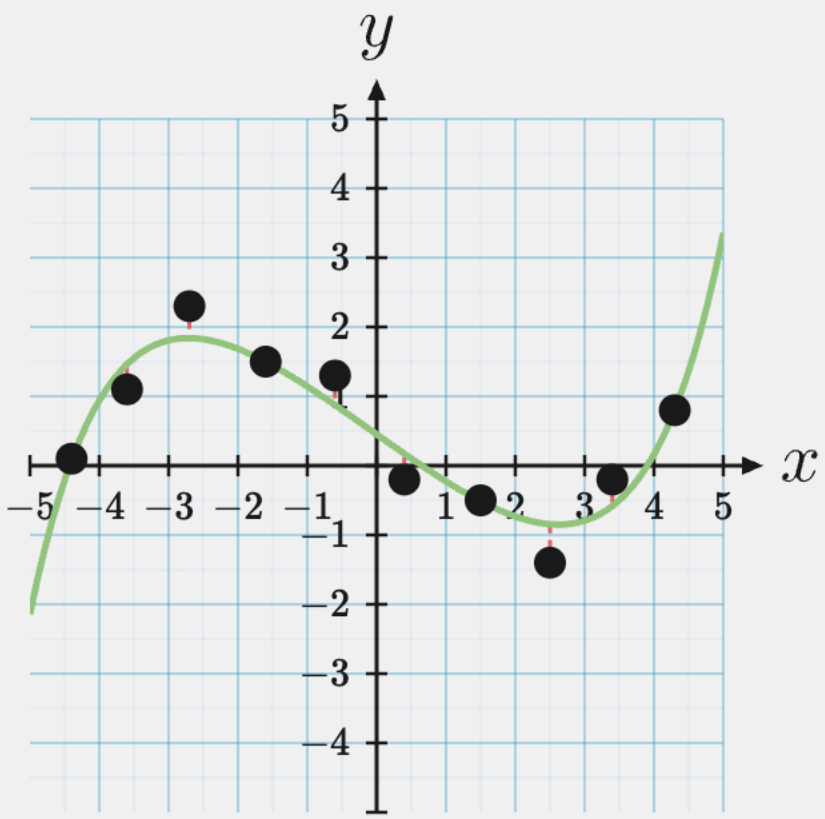
With the penalty $\lambda \|\theta\|_2^2$:

$$\theta_\lambda = (\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

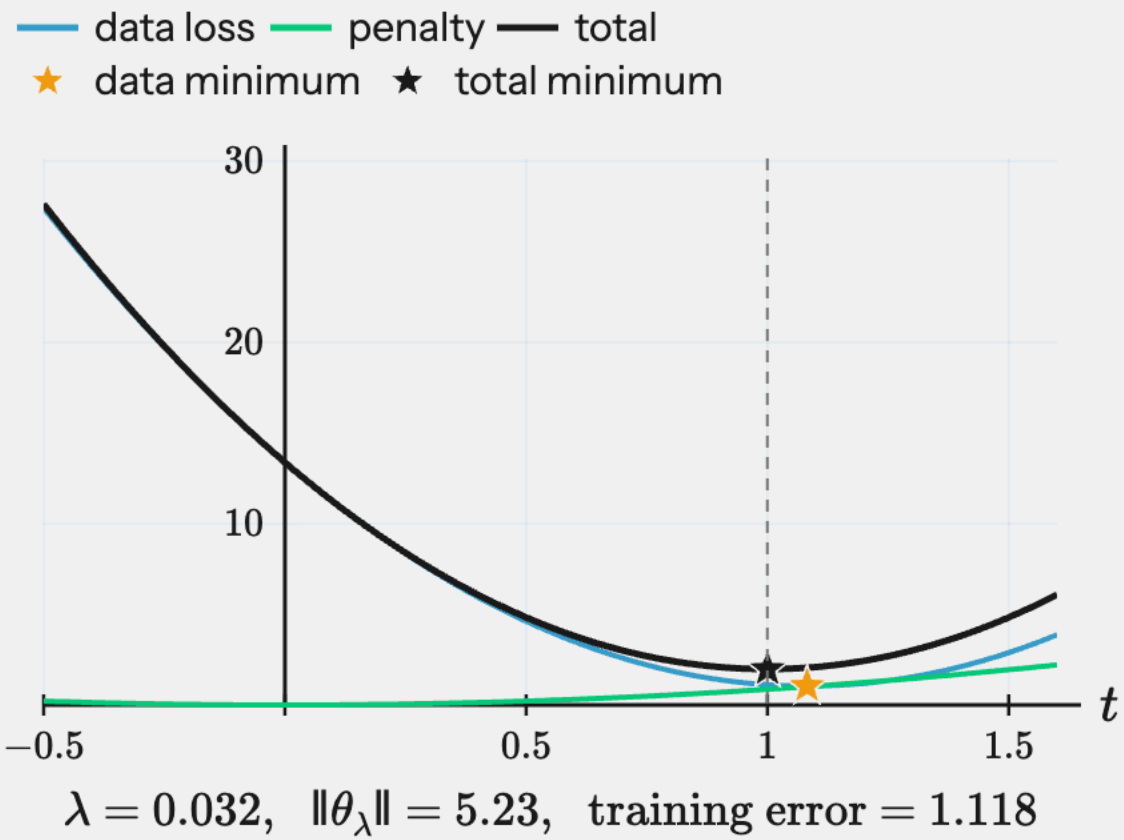
- $\lambda \boldsymbol{I}$ is the matrix form of λ: it adds λ to the diagonal of $\boldsymbol{X}^\top \boldsymbol{X}$.
- $\lambda = 0$ gives back OLS; larger λ shrinks θ_λ toward 0 (lecture 02's widget).

Ridge regression, live

$$\hat{y} = \phi_k(x)^\top \theta_\lambda$$

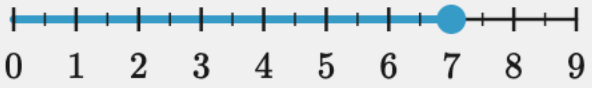


$$\mathcal{L}_{\text{reg}}(t\,\theta_\lambda, \lambda) \quad \text{along} \quad \theta = t\,\theta_\lambda$$

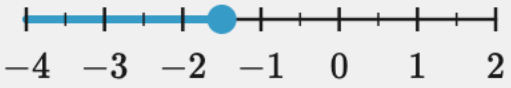


$\lambda = 0.032, \quad \|\theta_\lambda\| = 5.23, \quad \text{training error} = 1.118$

$k = 7$



$log_{10}\,\lambda = -1.50$



Advanced linear algebra

The *i*th canonical basis vector is defined as:

$$\boldsymbol{e}_i = [0, \ldots, 0, \underbrace{1}_{i\text{-th entry}}, 0, \ldots, 0]^\top \in \mathbb{R}^n$$

$$\boldsymbol{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \boldsymbol{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \qquad \boldsymbol{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$(n = 3)$

Advanced linear algebra

The **outer product** of two vectors is a matrix:

$$\boldsymbol{a} \in \mathbb{R}^n,\ \boldsymbol{b} \in \mathbb{R}^m : \qquad \boldsymbol{a}\boldsymbol{b}^\top \in \mathbb{R}^{n \times m}, \qquad (\boldsymbol{a}\boldsymbol{b}^\top)_{i,j} = a_i\, b_j$$

$$\sum_{i=1}^{3} \boldsymbol{e}_i \boldsymbol{e}_i^\top = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{\boldsymbol{e}_1 \boldsymbol{e}_1^\top} + \underbrace{\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{\boldsymbol{e}_2 \boldsymbol{e}_2^\top} + \underbrace{\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\boldsymbol{e}_3 \boldsymbol{e}_3^\top}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \boldsymbol{I} = [\,\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3\,]$$

Advanced linear algebra

The **inner product** of two vectors is a scalar:

$$\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^n : \qquad \boldsymbol{a}^\top \boldsymbol{b} = \sum_{i=1}^{n} a_i \, b_i \in \mathbb{R}$$

$$\begin{bmatrix} \boldsymbol{e}_1^\top \boldsymbol{e}_1 & \boldsymbol{e}_1^\top \boldsymbol{e}_2 & \boldsymbol{e}_1^\top \boldsymbol{e}_3 \\ \boldsymbol{e}_2^\top \boldsymbol{e}_1 & \boldsymbol{e}_2^\top \boldsymbol{e}_2 & \boldsymbol{e}_2^\top \boldsymbol{e}_3 \\ \boldsymbol{e}_3^\top \boldsymbol{e}_1 & \boldsymbol{e}_3^\top \boldsymbol{e}_2 & \boldsymbol{e}_3^\top \boldsymbol{e}_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \boldsymbol{I}$$

$$\delta_{ij} := \boldsymbol{e}_i^\top \boldsymbol{e}_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \qquad \text{so} \qquad I_{i,j} = \delta_{ij}$$

the **Kronecker delta**

$$\sum_j \delta_{ij} \, a_j = a_i$$

it isolates terms in a sum

Advanced linear algebra

The **diag** function puts a vector on the diagonal of a matrix:

$$\operatorname{diag}(\boldsymbol{a}) = \begin{bmatrix} a_1 & 0 & \cdots & 0 \\ 0 & a_2 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & a_n \end{bmatrix} \in \mathbb{R}^{n \times n}, \qquad \operatorname{diag}(\boldsymbol{a})_{i,j} = a_i \, \delta_{ij}$$

$$\operatorname{diag}(\mathbf{1}) = \operatorname{diag}\left(\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \boldsymbol{I}$$

Advanced linear algebra

The **Hadamard product** of two matrices (or vectors) of the same shape is their element-wise product:

$$(\boldsymbol{A} \circ \boldsymbol{B})_{i,j} = A_{i,j}\, B_{i,j}, \qquad (\boldsymbol{a} \circ \boldsymbol{b})_i = a_i\, b_i$$

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \circ \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} a_1 b_1 \\ a_2 b_2 \\ a_3 b_3 \end{bmatrix} = \begin{bmatrix} a_1 & 0 & 0 \\ 0 & a_2 & 0 \\ 0 & 0 & a_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$\boldsymbol{a} \circ \boldsymbol{b} = \operatorname{diag}(\boldsymbol{a})\, \boldsymbol{b}$$

Advanced vector calculus: some new notation

Differentiating an **element-wise non-linearity** $f: \mathbb{R} \to \mathbb{R}$, applied to each entry of a vector:

$$\boldsymbol{z} = f(\boldsymbol{x}), \qquad z_i = f(x_i), \qquad f : \mathbb{R}^n \to \mathbb{R}^n \quad \Rightarrow \quad \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} \in \mathbb{R}^{n \times n}$$

input and output in $\mathbb{R}^n$: we are looking for a matrix

$$\frac{\partial z_i}{\partial x_j} = f'(x_i)\, \delta_{ij}$$

z_i depends only on x_i: single-variable calculus

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \begin{bmatrix} f'(x_1) & 0 & \cdots & 0 \\ 0 & f'(x_2) & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & f'(x_n) \end{bmatrix} = \operatorname{diag}\left(f'(\boldsymbol{x})\right)$$

only the diagonal survives

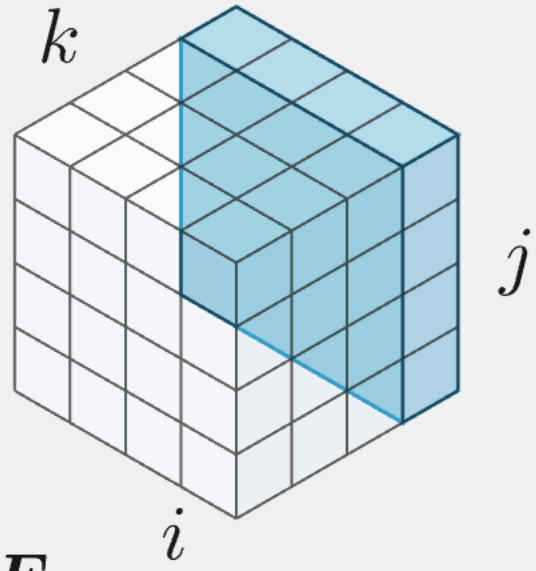
Advanced vector calculus: some new notation

Differentiating the **outer product**: the input is in $\mathbb{R}^n$, the output in $\mathbb{R}^{n\times n}$.

$$\boldsymbol{F} = \boldsymbol{x}\boldsymbol{x}^\top, \qquad F_{i,j} = x_i\, x_j, \qquad \boldsymbol{x} \in \mathbb{R}^n, \quad \boldsymbol{F} \in \mathbb{R}^{n\times n}$$

$$\frac{\partial F_{i,j}}{\partial x_k} = \delta_{ik}\, x_j + x_i\, \delta_{jk}$$

$$\frac{\partial \boldsymbol{F}}{\partial \boldsymbol{x}} \in \mathbb{R}^{n\times n\times n}$$



$$\frac{\partial \boldsymbol{F}}{\partial x_k} = \boldsymbol{e}_k\boldsymbol{x}^\top + \boldsymbol{x}\,\boldsymbol{e}_k^\top$$

product rule on the number $x_i x_j$, differentiating with respect to an arbitrary x_k; $\partial x_i/\partial x_k = \delta_{ik}$

three free indices i, j, k: a 3-tensor, which cannot be reassembled into a matrix

Recalling our generalized linear model

Generalized Linear Model (GLM):

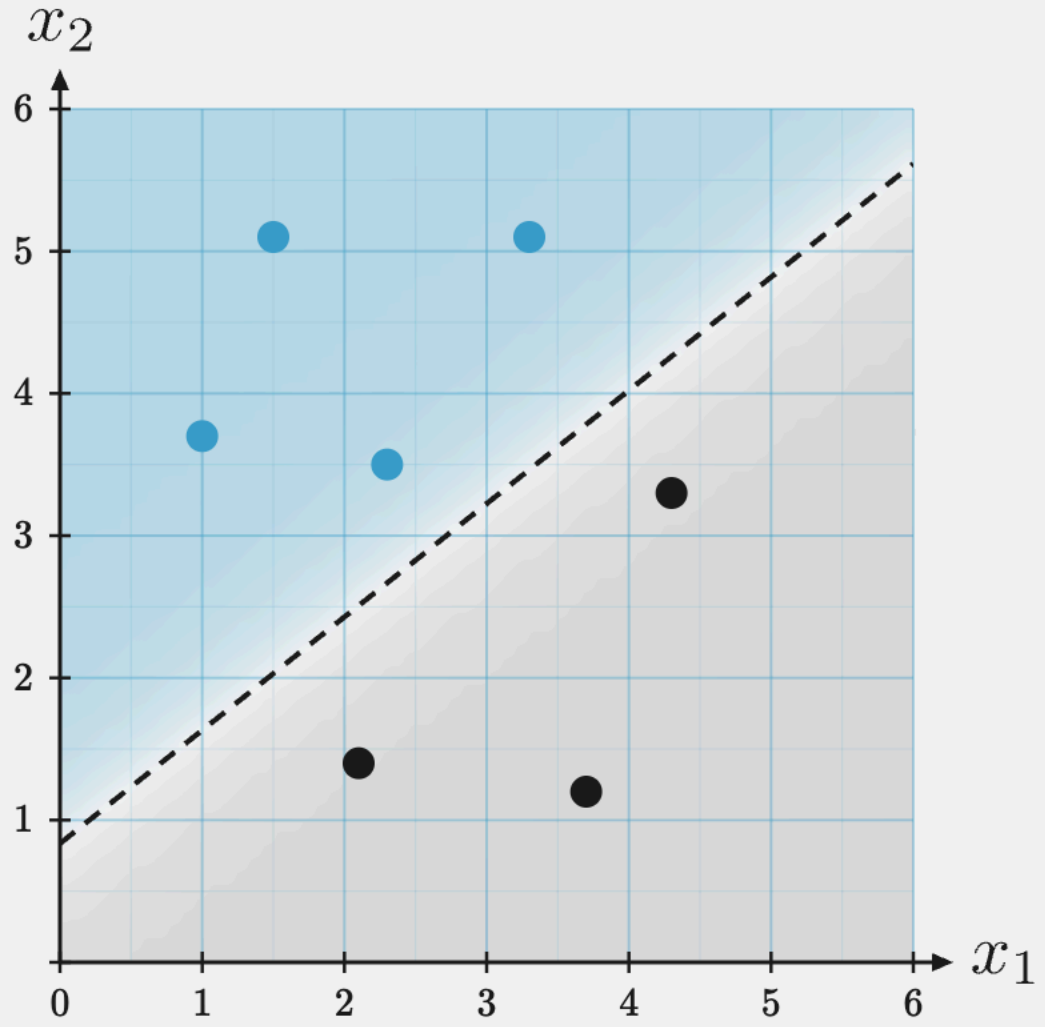
$$\hat{y} = g^{-1}(\theta^\top \boldsymbol{x})$$

GLM with basis expansion:

$$\hat{y} = g^{-1}\big(\theta^\top \phi(\boldsymbol{x})\big)$$

The function *g* is called the link function and is generally associated with an exponential distribution (to be covered).

Logistic regression



$$\hat{y} = g^{-1}\big(\theta^\top \phi(\boldsymbol{x})\big) = \sigma\big(\theta^\top \phi(\boldsymbol{x})\big)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \phi(\boldsymbol{x}) = [x_1, x_2, 1]^\top$$

$$\mathcal{L}(\theta) = -\sum_{i=1}^{m} \Big(y_i \log s_i + (1-y_i)\log(1-s_i) \Big)$$

$$s_i = \sigma\big(\theta^\top \phi(\boldsymbol{x}_i)\big)$$

$$\textbf{iteration } 124, \quad \mathcal{L}(\theta) = 0.045$$

$$\theta = [-1.92,\ 2.41,\ -2.02]^\top$$

θ found by gradient descent: later in this lecture

Differentiating logistic regression with CE loss

For a design matrix $\boldsymbol{X} \in \mathbb{R}^{m \times n}$ (no basis expansion here, to keep the shapes simple) and labels $\boldsymbol{y} \in \{0, 1\}^m$, we can write the cross-entropy loss in vector form, with log and σ applied element-wise and $\mathbf{1}$ the vector of ones, as a composition of three functions:

$$\mathcal{L}(\theta) = \underbrace{-\left(\boldsymbol{y}^\top \log(\boldsymbol{s}) + (\mathbf{1} - \boldsymbol{y})^\top \log(\mathbf{1} - \boldsymbol{s})\right)}_{\text{outer}}, \qquad \boldsymbol{s} = \underbrace{\sigma(\boldsymbol{z})}_{\text{middle}}, \qquad \boldsymbol{z} = \underbrace{\boldsymbol{X}\theta}_{\text{inner}}$$

$$\theta \longmapsto \boldsymbol{z} \longmapsto \boldsymbol{s} \longmapsto \mathcal{L}$$

$$\nabla_\theta \mathcal{L} = \underbrace{\nabla_{\boldsymbol{s}} \mathcal{L}}_{(1 \times m)} \cdot \underbrace{\nabla_{\boldsymbol{z}} \boldsymbol{s}}_{(m \times m)} \cdot \underbrace{\nabla_\theta \boldsymbol{z}}_{(m \times n)}$$

a $(1 \times m)(m \times m)(m \times n)$ product: we compute each factor in turn

The inner factor

inner: $\boldsymbol{z} = \boldsymbol{X}\theta$, a linear function of θ

$$\underbrace{\boldsymbol{z}}_{m\times 1} = \underbrace{\boldsymbol{X}}_{m\times n}\underbrace{\theta}_{n\times 1}$$

$$z_i = \boldsymbol{X}_{i,:}\,\theta = \sum_k X_{i,k}\,\theta_k$$

one entry: row i of $\boldsymbol{X}$ times θ, a sum over the n parameters

$$\frac{\partial z_i}{\partial \theta_k} = X_{i,k}$$

only one term of the sum contains θ_k, and its coefficient is $X_{i,k}$

$$\underbrace{\nabla_\theta \boldsymbol{z}}_{m\times n} = \boldsymbol{X}$$

entry (i, k) of the Jacobian is $X_{i,k}$: the Jacobian is $\boldsymbol{X}$ itself (as $\nabla_\theta \boldsymbol{c}^\top\theta = \boldsymbol{c}^\top$ in lecture 02, one row at a time)

The middle factor

middle: $\boldsymbol{s} = \sigma(\boldsymbol{z})$: each s_i depends only on its own z_i, and the logistic function has the convenient derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

$$s_i = \sigma(z_i): \qquad \frac{\partial s_i}{\partial z_i} = \sigma'(z_i) = s_i(1 - s_i), \qquad \frac{\partial s_i}{\partial z_j} = 0 \;\; (j \neq i)$$

$$\underbrace{\nabla_{\boldsymbol{z}} \boldsymbol{s}}_{m \times m} = \begin{bmatrix} s_1(1 - s_1) & & \\ & \ddots & \\ & & s_m(1 - s_m) \end{bmatrix} = \operatorname{diag}\left(\boldsymbol{s} \circ (\mathbf{1} - \boldsymbol{s})\right)$$

so the $m \times m$ Jacobian is zero off the diagonal (the element-wise rule of slide 16, with $f = \sigma$)

The outer factor

outer: the cross-entropy, as a function of $\boldsymbol{s}$

$$\mathcal{L}(\boldsymbol{s}) = -\big(\boldsymbol{y}^\top \log(\boldsymbol{s}) + (\mathbf{1}-\boldsymbol{y})^\top \log(\mathbf{1}-\boldsymbol{s})\big)$$

$$\boldsymbol{y}^\top \log(\boldsymbol{s}) = \sum_{i=1}^{m} y_i \log(s_i) = y_1 \log(s_1) + \cdots + y_j \log(s_j) + \cdots + y_m \log(s_m)$$

$$\frac{\partial}{\partial s_j} \boldsymbol{y}^\top \log(\boldsymbol{s}) = y_j \cdot \frac{1}{s_j} = \frac{y_j}{s_j}$$

$$\frac{\partial}{\partial s_j} (\mathbf{1}-\boldsymbol{y})^\top \log(\mathbf{1}-\boldsymbol{s}) = (1-y_j) \cdot \frac{1}{1-s_j} \cdot (-1) = -\frac{1-y_j}{1-s_j}$$

$$\frac{\partial \mathcal{L}}{\partial s_j} = -\left(\frac{y_j}{s_j} - \frac{1-y_j}{1-s_j}\right)$$

sum rule; the minus sign in front is part of the definition of the loss

Combining the factors

$$\underbrace{\nabla_{\boldsymbol{s}} \mathcal{L}}_{1 \times m} = \left[\ldots, -\left(\frac{y_j}{s_j} - \frac{1 - y_j}{1 - s_j} \right), \ldots \right]$$

$$\underbrace{\nabla_{\boldsymbol{z}} \boldsymbol{s}}_{m \times m} = \operatorname{diag}\left(\boldsymbol{s} \circ (\mathbf{1} - \boldsymbol{s}) \right)$$

$$\underbrace{\nabla_{\theta} \boldsymbol{z}}_{m \times n} = \boldsymbol{X}$$

$$\nabla_{\theta} \mathcal{L} = \nabla_{\boldsymbol{s}} \mathcal{L} \, \nabla_{\boldsymbol{z}} \boldsymbol{s} \, \nabla_{\theta} \boldsymbol{z}$$

$$\nabla_{\boldsymbol{s}} \mathcal{L} \, \nabla_{\boldsymbol{z}} \boldsymbol{s} = \underbrace{(\boldsymbol{s} - \boldsymbol{y})^{\top}}_{1 \times m}$$

$$\nabla_{\theta} \mathcal{L} = \underbrace{(\boldsymbol{s} - \boldsymbol{y})^{\top} \boldsymbol{X}}_{1 \times n} = \left(\sigma(\boldsymbol{X}\theta) - \boldsymbol{y} \right)^{\top} \boldsymbol{X}$$

recall the three factors: outer (slide 23), middle (slide 22), inner (slide 21)

the chain rule: multiply them in order

outer times middle: the fractions cancel, leaving $s_j - y_j$ in entry j (a practice problem)

times the inner factor: the full gradient

The Hessian

The *Hessian* of a scalar function $\mathcal{L}: \mathbb{R}^n \to \mathbb{R}$ is the matrix of its second partial derivatives:

$$(\nabla^2_\theta \mathcal{L})_{j,k} = \frac{\partial^2 \mathcal{L}}{\partial \theta_j \, \partial \theta_k}$$

entry (j, k): differentiate with respect to θ_j, then θ_k

$$\nabla^2_\theta \mathcal{L} = \begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial \theta_1 \, \partial \theta_1} & \cdots & \frac{\partial^2 \mathcal{L}}{\partial \theta_1 \, \partial \theta_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 \mathcal{L}}{\partial \theta_n \, \partial \theta_1} & \cdots & \frac{\partial^2 \mathcal{L}}{\partial \theta_n \, \partial \theta_n} \end{bmatrix}$$

one row and one column per parameter: $n \times n$

$$\underbrace{\nabla^2_\theta \mathcal{L}}_{n \times n} = \nabla_\theta \underbrace{(\nabla_\theta \mathcal{L})^\top}_{n \times 1}$$

equivalently: transpose the gradient (a row vector) to a column, and differentiate it once more

The Hessian of logistic regression

Recall the gradient of logistic regression (slide 24), transposed to a column:

$$\nabla_\theta \mathcal{L} = \big(\sigma(\boldsymbol{X}\theta) - \boldsymbol{y}\big)^\top \boldsymbol{X} \qquad \Longrightarrow \qquad (\nabla_\theta \mathcal{L})^\top = \boldsymbol{X}^\top \big(\sigma(\boldsymbol{X}\theta) - \boldsymbol{y}\big)$$

$$(\nabla_\theta \mathcal{L})^\top = \underbrace{\boldsymbol{X}^\top (\boldsymbol{s} - \boldsymbol{y})}_{\text{outer}}, \qquad \boldsymbol{s} = \underbrace{\sigma(\boldsymbol{z})}_{\text{middle}}, \qquad \boldsymbol{z} = \underbrace{\boldsymbol{X}\theta}_{\text{inner}}$$

Recall, we want to differentiate this again with respect to the parameters $\theta \in \mathbb{R}^n$. A column of n entries, each differentiated with respect to n parameters, gives a Hessian of shape $n \times n$.

$$\underbrace{\nabla^2_\theta \mathcal{L}}_{n \times n} = \underbrace{\nabla_{\boldsymbol{s}} \big(\boldsymbol{X}^\top (\boldsymbol{s} - \boldsymbol{y})\big)}_{n \times m} \cdot \underbrace{\nabla_{\boldsymbol{z}} \boldsymbol{s}}_{m \times m} \cdot \underbrace{\nabla_\theta \boldsymbol{z}}_{m \times n}$$

the same composition as for the gradient, so the same chain rule

The Hessian: the outer factor

outer: $\boldsymbol{g}(\boldsymbol{s}) = \boldsymbol{X}^\top(\boldsymbol{s} - \boldsymbol{y})$, as a function of $\boldsymbol{s}$ ($\boldsymbol{X}$ and $\boldsymbol{y}$ are data: constants)

$$\boldsymbol{g}(\boldsymbol{s}) = \boldsymbol{X}^\top(\boldsymbol{s} - \boldsymbol{y}) = \boldsymbol{X}^\top\boldsymbol{s} - \boldsymbol{X}^\top\boldsymbol{y}$$

multiply out

$$\nabla_{\boldsymbol{s}}\boldsymbol{g} = \nabla_{\boldsymbol{s}}\left(\boldsymbol{X}^\top\boldsymbol{s} - \boldsymbol{X}^\top\boldsymbol{y}\right) = \nabla_{\boldsymbol{s}}\left(\boldsymbol{X}^\top\boldsymbol{s}\right)$$

$\boldsymbol{X}^\top\boldsymbol{y}$ does not depend on $\boldsymbol{s}$: its derivative is zero

$$= \underbrace{\boldsymbol{X}^\top}_{n \times m}$$

the rule we derived for the inner factor, $\nabla_\theta(\boldsymbol{X}\theta) = \boldsymbol{X}$ (slide 21), with $\boldsymbol{X}^\top$ in place of $\boldsymbol{X}$

The Hessian: putting it together

$$\nabla^2_\theta \mathcal{L} = \nabla_{\boldsymbol{s}}\left(\boldsymbol{X}^\top(\boldsymbol{s}-\boldsymbol{y})\right) \cdot \nabla_{\boldsymbol{z}}\boldsymbol{s} \cdot \nabla_\theta \boldsymbol{z}$$

the chain rule (slide 26)

$$= \underbrace{\boldsymbol{X}^\top}_{n\times m} \underbrace{\operatorname{diag}\left(\boldsymbol{s}\circ(\boldsymbol{1}-\boldsymbol{s})\right)}_{m\times m} \underbrace{\boldsymbol{X}}_{m\times n}$$

outer (slide 27), middle (slide 22), inner (slide 21)

$$= \underbrace{\boldsymbol{X}^\top \boldsymbol{S} \boldsymbol{X}}_{n\times n}, \qquad \boldsymbol{S} := \operatorname{diag}\left(\boldsymbol{s}\circ(\boldsymbol{1}-\boldsymbol{s})\right)$$

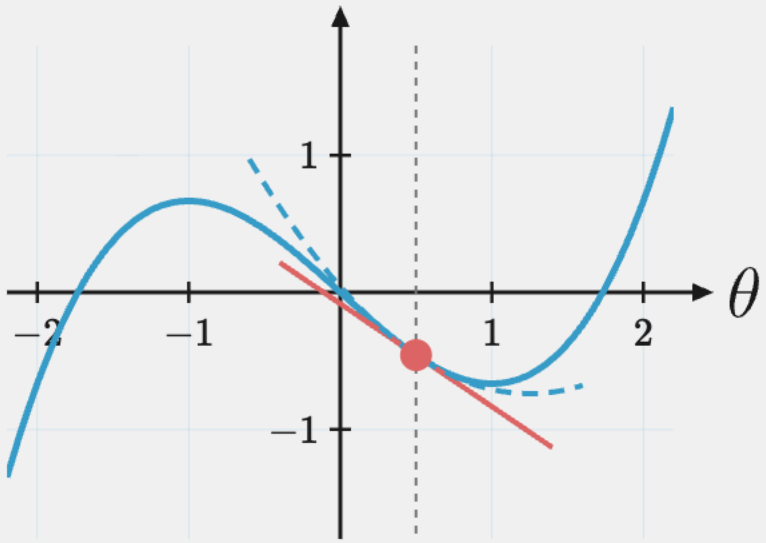
name the diagonal matrix ***S***

$$(\nabla^2_\theta \mathcal{L})_{j,k} = \sum_{i=1}^{m} X_{i,j}\; s_i(1-s_i)\; X_{i,k}$$

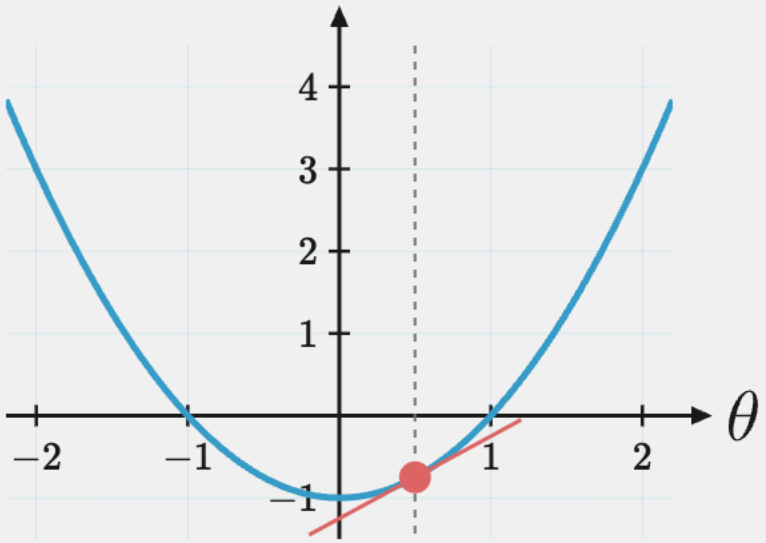
entry by entry: the labels ***y*** do not appear

Visualizing the gradient and the Hessian for an arbitrary function

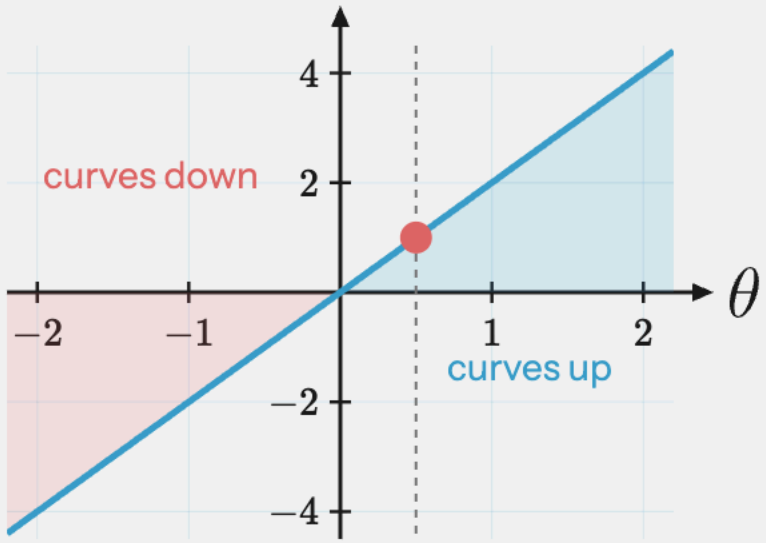
$$f(\theta) = \frac{1}{3}\theta^3 - \theta$$



$$\nabla_\theta f = \theta^2 - 1$$



$$\nabla^2_\theta f = 2\theta$$



$\theta = 0.5 \quad f(\theta) = -0.46 \quad \nabla f = -0.75 \quad \nabla^2 f = 1$

$\theta = 0.50$



Hessian positive: the function curves up (a local bowl)

Recalling that direct solution fails

We have computed the gradient of logistic regression to be:

$$\nabla_\theta \mathcal{L} = \left(\sigma(\boldsymbol{X}\theta) - \boldsymbol{y}\right)^\top \boldsymbol{X}$$

Because we cannot solve, we need to figure out a new step 3!

But we can still do the first two steps!

1. Expand our loss function
2. Take gradient of loss function
3. Set gradient equal to zero, solve for theta

Learning algorithm: gradient descent

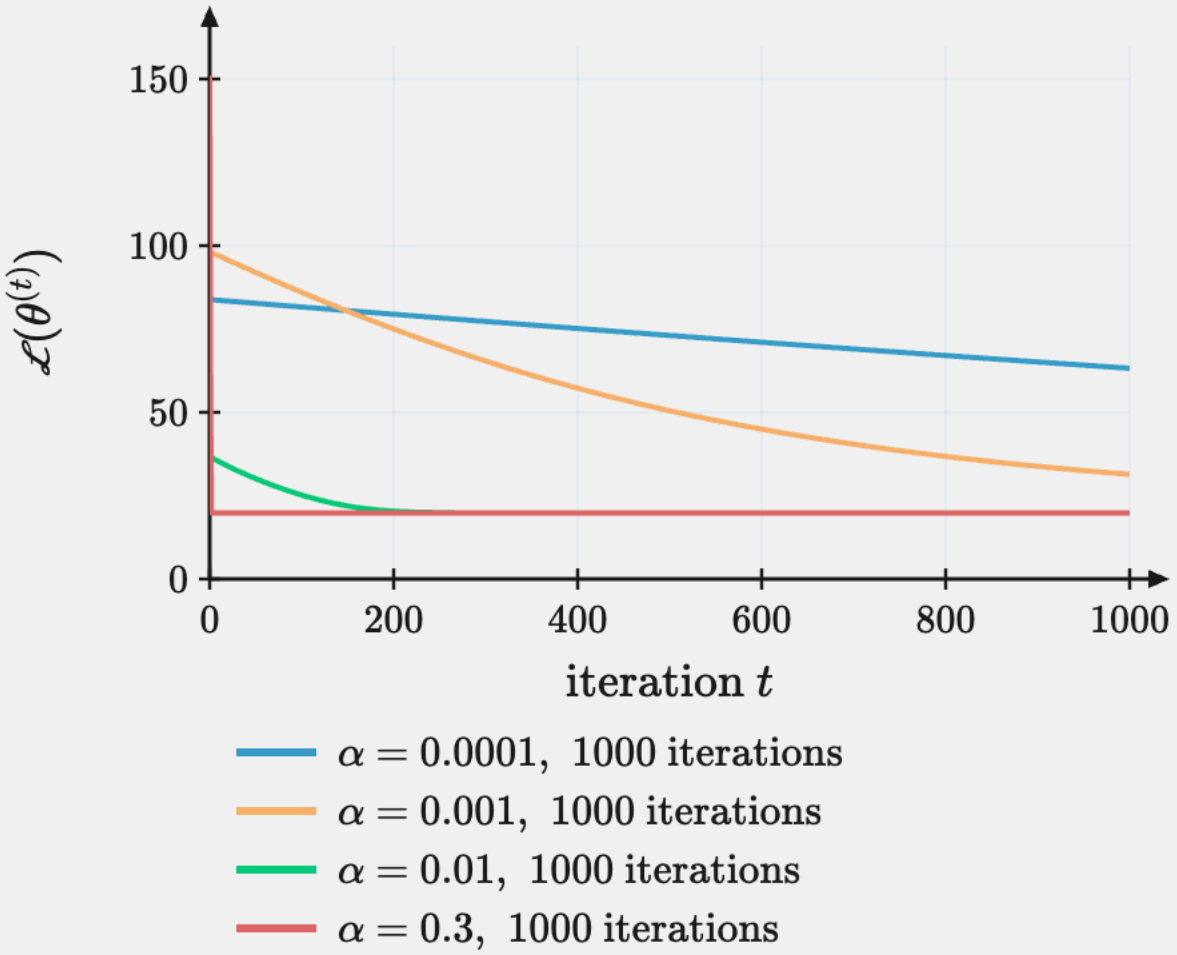
$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}\left(\theta^{(t)}\right)^\top$$

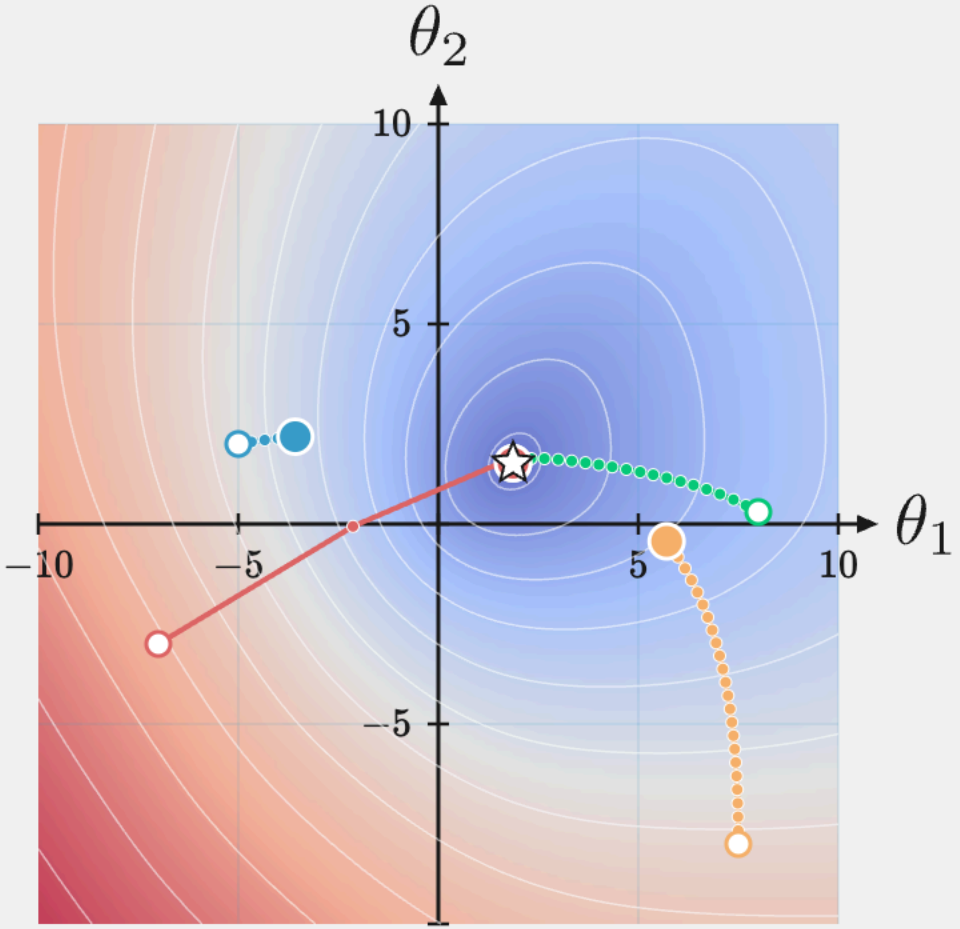
α: the *learning rate* (step size)

Gradient Descent Algorithm:

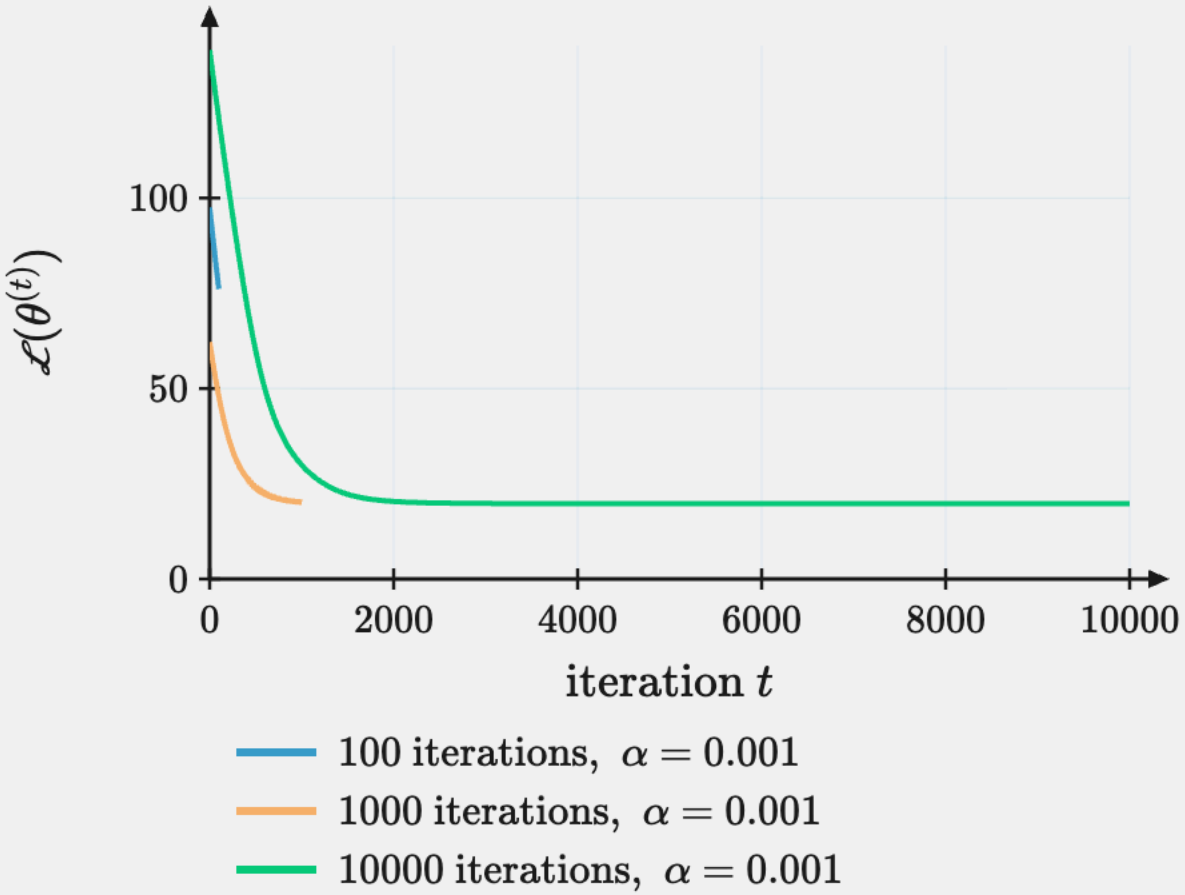
1. Randomly initialize our model
2. Take gradient of loss function w/data
3. Update model parameters w/grad
4. Repeat 2 and 3 'k' times

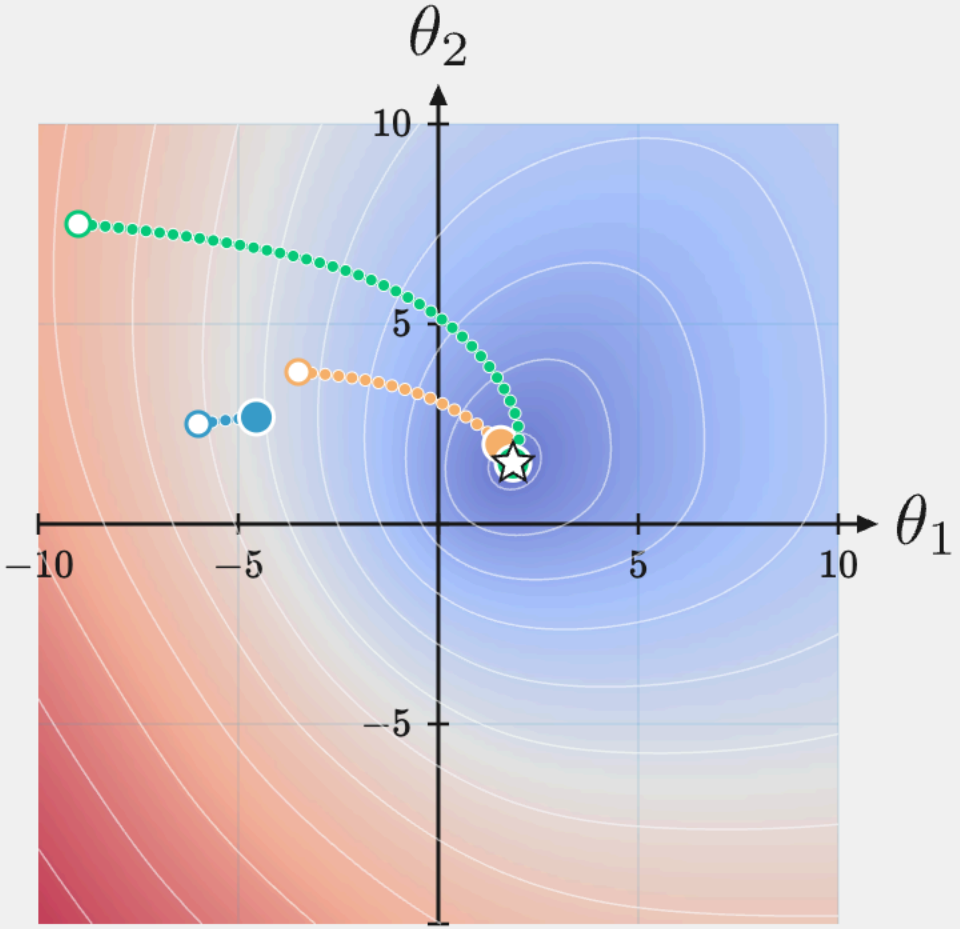
Effect of the learning rate



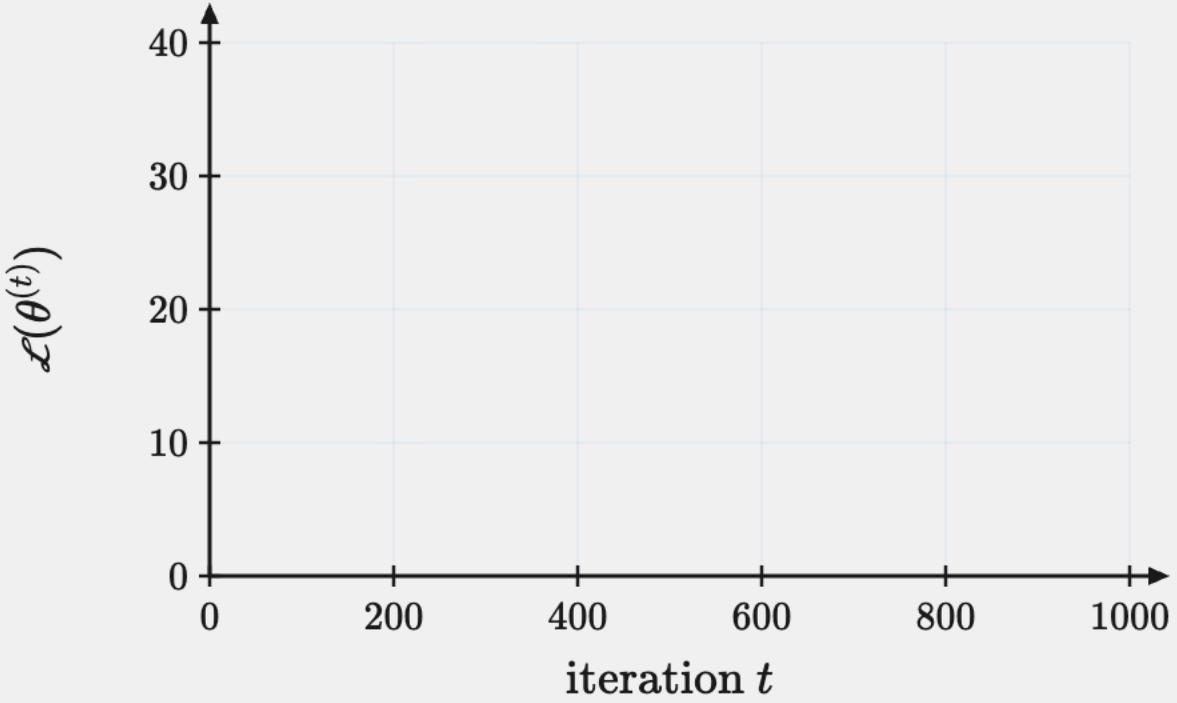


Effect of the iterations/epochs



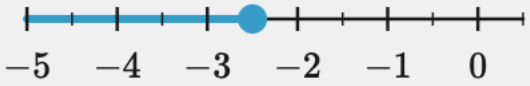


Effect of the iterations/epochs



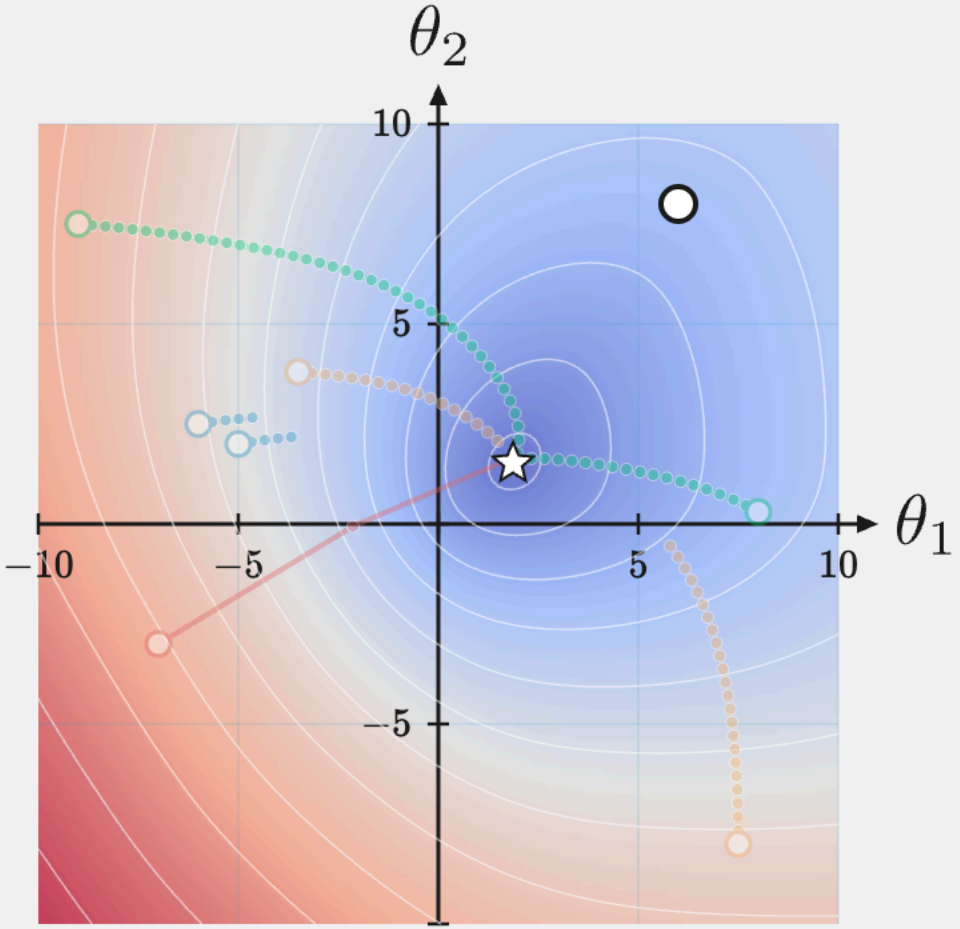
$\alpha = 0.0032, \; k = 1000$

$log_{10}\, \alpha = -2.50$



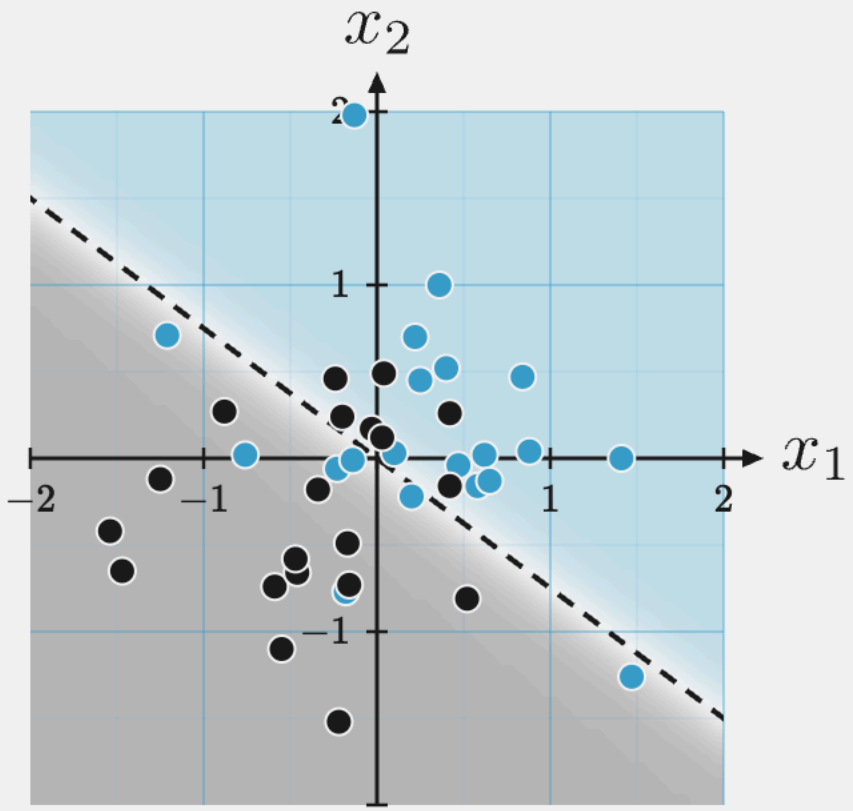
$log_{10}\, k = 3.0$



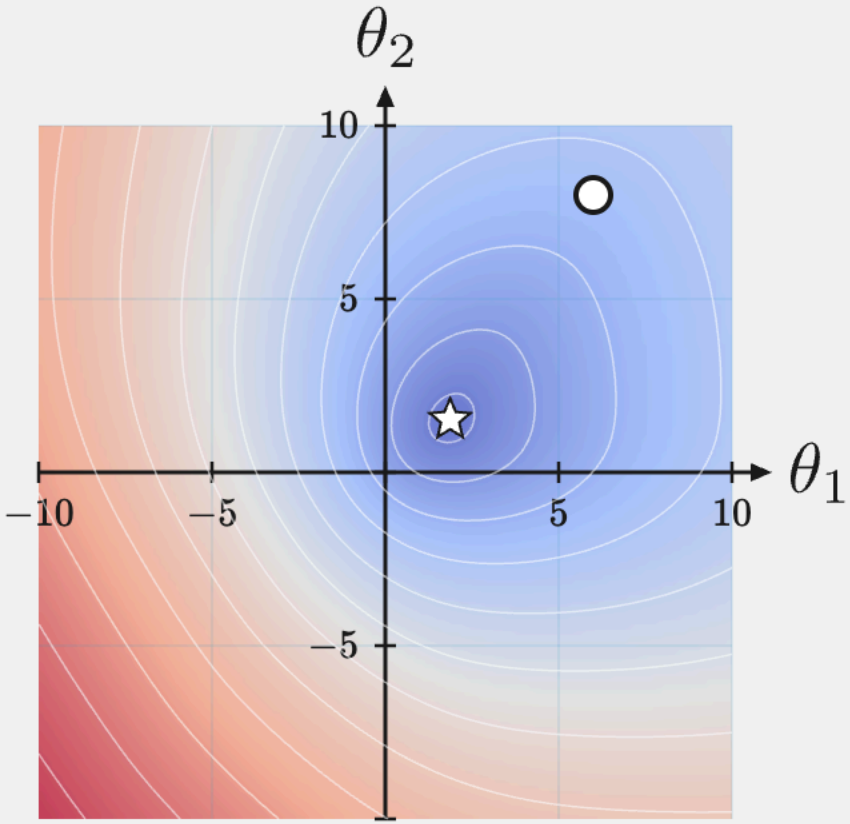


Learning in function space

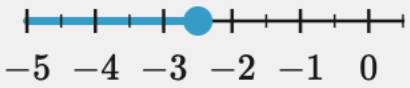
function space: drag or click the data



parameter space



$log_{10}\,\alpha = -2.50$



$log_{10}\,k = 3.0$



$t = 0,\ \theta = (6, 8),\ \mathcal{L} = 36.6$

$\alpha = 0.0032,\ k = 1000$

Gradient descent or OLS?

For linear models, should we use gradient descent or should we use OLS?

While there are many perspectives we could take to start to answer this question, we will begin by looking at computational complexity.

Computational Complexity of GD

$$(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

$$\theta^{(t)} - \alpha \left(\nabla_\theta \mathcal{L}\right)^\top$$

Computational Complexity of GD

$$\underbrace{(\boldsymbol{X}^\top \boldsymbol{X})^{-1}}\boldsymbol{X}^\top \boldsymbol{y}$$

Matrix product complexity:

$$\boldsymbol{C}_{i,j} = \sum_{k=1}^{l} \boldsymbol{A}_{i,k}\boldsymbol{B}_{k,j}$$

$$\boldsymbol{A} \in \mathbb{R}^{n\times l}, \quad \boldsymbol{B} \in \mathbb{R}^{l\times m}$$

$$\theta^{(t)} - \alpha \left(\nabla_\theta \mathcal{L}\right)^\top$$

Computational Complexity of GD

$$\underbrace{(\boldsymbol{X}^\top \boldsymbol{X})^{-1}}\boldsymbol{X}^\top \boldsymbol{y}$$

Matrix product complexity:

$$\boldsymbol{C}_{i,j} = \sum_{k=1}^{l} \boldsymbol{A}_{i,k} \boldsymbol{B}_{k,j}$$

$$\boldsymbol{A} \in \mathbb{R}^{n \times l}, \quad \boldsymbol{B} \in \mathbb{R}^{l \times m}$$

```
for i ∈ [1, ..., n]
  for j ∈ [1, ..., m]
    sum A[i][k] B[k][j],  k ∈ [1, ..., l]
```

$\Rightarrow \quad O(nml) \qquad \boldsymbol{X}^\top \boldsymbol{X}: \quad O(n^2 m)$

$$\theta^{(t)} - \alpha \left(\nabla_\theta \mathcal{L}\right)^\top$$

Computational Complexity of GD

$$\underbrace{(\boldsymbol{X}^\top \boldsymbol{X})^{-1}}\boldsymbol{X}^\top \boldsymbol{y}$$

Matrix inverse complexity:

There are several algorithms for matrix inversion, and one can do slightly better than cubic, but in general we will take:

$$O(n^3)$$

$$\theta^{(t)} - \alpha \left(\nabla_\theta \mathcal{L}\right)^\top$$

Computational Complexity of GD

$$(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Overall complexity:

$$O(n^2 m + n^3)$$

$$\theta^{(t)} - \alpha \left(\nabla_\theta \mathcal{L}\right)^\top$$

Computational Complexity of GD

$$(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Overall complexity:

$$O(n^2 m + n^3)$$

$$\theta^{(t)} - \alpha \underbrace{(\nabla_\theta \mathcal{L})^\top}$$

$$2\big(\underbrace{(\boldsymbol{X}^\top \boldsymbol{X})}\theta - \boldsymbol{X}^\top \boldsymbol{y}\big)$$

$$\boldsymbol{X} \in \mathbb{R}^{m \times n}$$

$$O(n^2 m)$$

Computational Complexity of GD

$$(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Overall complexity:

$$O(n^2 m + n^3)$$

$$\theta^{(t)} - \alpha \underbrace{(\nabla_\theta \mathcal{L})^\top}$$

$$2\big(\underbrace{(\boldsymbol{X}^\top \boldsymbol{X})}\theta - \boldsymbol{X}^\top \boldsymbol{y}\big)$$

$$\boldsymbol{X} \in \mathbb{R}^{m \times n}$$

$$O(n^2 m)$$

However, this is really inefficient!

Rewriting the gradient

The same gradient, written so that we never form $\boldsymbol{X}^\top\boldsymbol{X}$:

$$(\nabla_\theta \mathcal{L})^\top = 2\big((\boldsymbol{X}^\top\boldsymbol{X})\theta - \boldsymbol{X}^\top\boldsymbol{y}\big)$$

as on the previous slide: $\boldsymbol{X}^\top\boldsymbol{X}$ costs $O(n^2m)$

$$= 2\big(\boldsymbol{X}^\top(\boldsymbol{X}\theta) - \boldsymbol{X}^\top\boldsymbol{y}\big)$$

matrix products are associative: $(\boldsymbol{X}^\top\boldsymbol{X})\theta = \boldsymbol{X}^\top(\boldsymbol{X}\theta)$

$$= 2\boldsymbol{X}^\top(\boldsymbol{X}\theta - \boldsymbol{y})$$

factor out $\boldsymbol{X}^\top$ from both terms (distributivity)

> The same vector, but the brackets change the order of the multiplications, and so the cost.

Computational Complexity of GD

$$(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Overall complexity:

$$O(n^2 m + n^3)$$

$$\theta^{(t)} - \alpha \underbrace{(\nabla_\theta \mathcal{L})^\top}_{\underbrace{2\boldsymbol{X}^\top(\underbrace{\boldsymbol{X}\theta}_{O(mn)} - \boldsymbol{y})}_{O(mn)}}$$

Computational Complexity of GD

$$(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Overall complexity:

$$O(n^2 m + n^3)$$

$$\theta^{(t)} - \alpha \underbrace{(\nabla_\theta \mathcal{L})^\top}_{\underbrace{2\boldsymbol{X}^\top (\underbrace{\boldsymbol{X}\theta}_{O(mn)} - \boldsymbol{y})}_{O(mn)}}$$

k steps: $O(kmn)$

Computational Complexity of GD

$$(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Overall complexity:

$$O(n^2 m + n^3)$$

$$\theta^{(t)} - \alpha \underbrace{(\nabla_\theta \mathcal{L})^\top}$$

Overall complexity:

$$O(kmn)$$

Computational Complexity of GD

$$(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Overall complexity:

$$O(n^2 m + n^3)$$

$$\theta^{(t)} - \alpha \underbrace{(\nabla_\theta \mathcal{L})^\top}$$

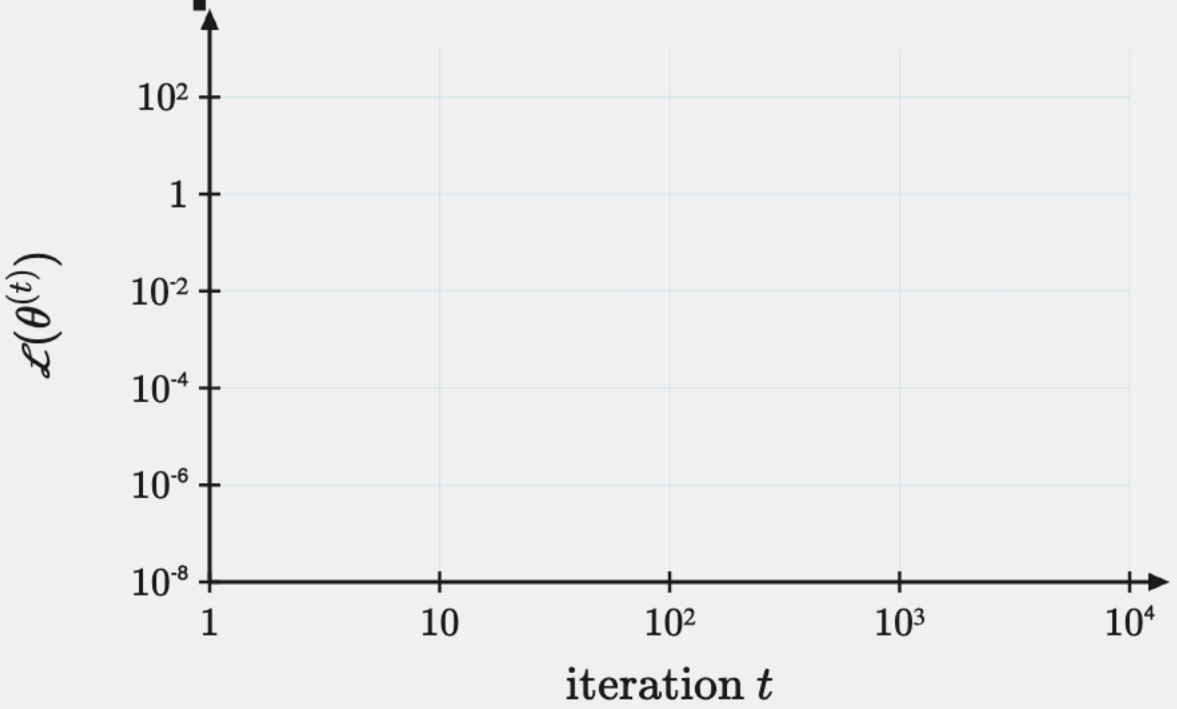
Overall complexity:

$$O(kmn)$$

$$\downarrow \text{(S)GD}$$

$$O(kbn)$$

Where gradient descent may need many steps



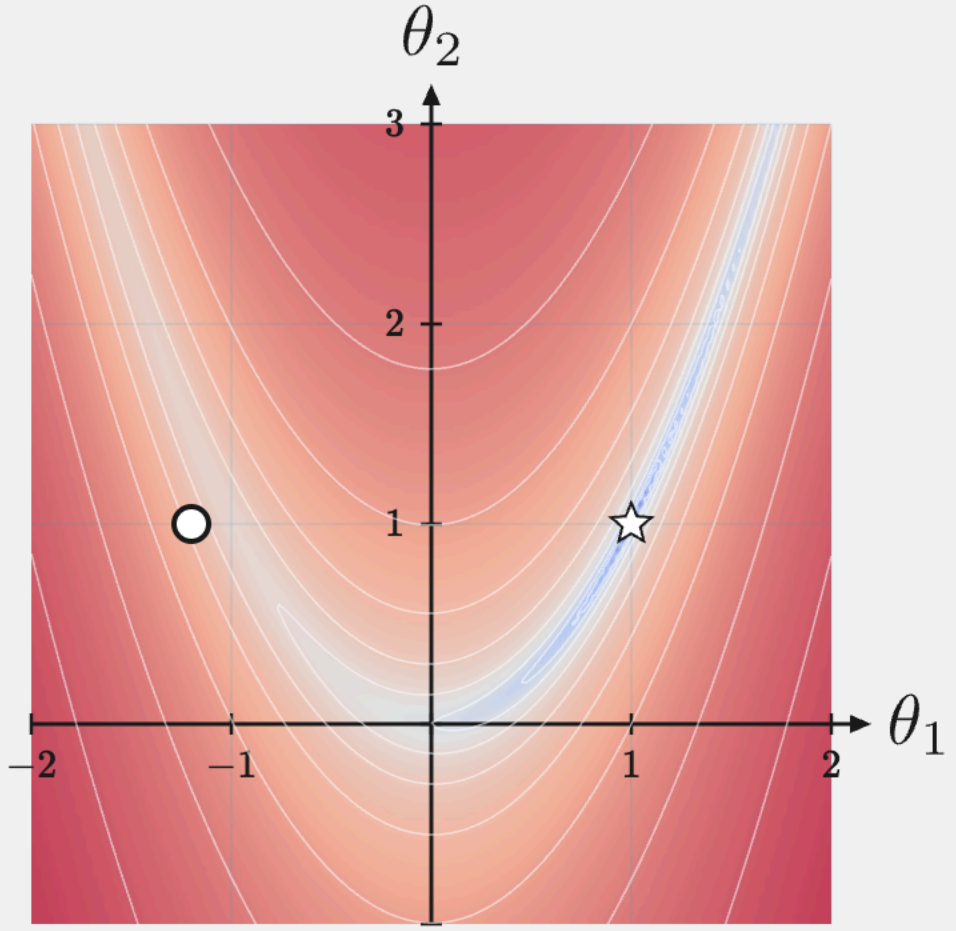
$\alpha = 0.001, \; k = 10000$

$log_{10}\, \alpha = -3.00$



$log_{10}\, k = 4.0$





Newton's Method

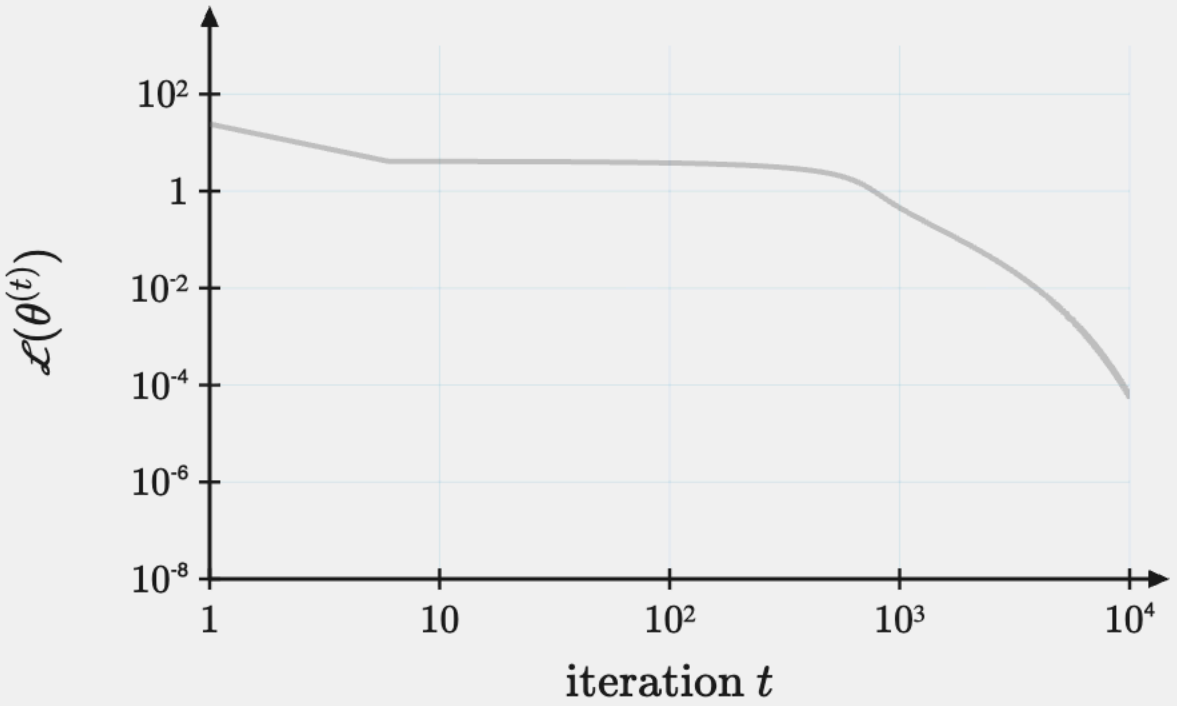
Instead of a fixed learning rate, scale each step by the curvature of the loss, the Hessian:

gradient descent: $\theta^{(t+1)} = \theta^{(t)} - \alpha \left(\nabla_\theta \mathcal{L}\right)^\top$

Newton's method: $\theta^{(t+1)} = \theta^{(t)} - \left(\nabla^2_\theta \mathcal{L}\right)^{-1} \left(\nabla_\theta \mathcal{L}\right)^\top$

> By scaling our steps by the curvature of the loss landscape we are able to
> (1) get rid of the learning-rate hyper-parameter α, and
> (2) converge much more quickly.

Visualizing Newton's Method



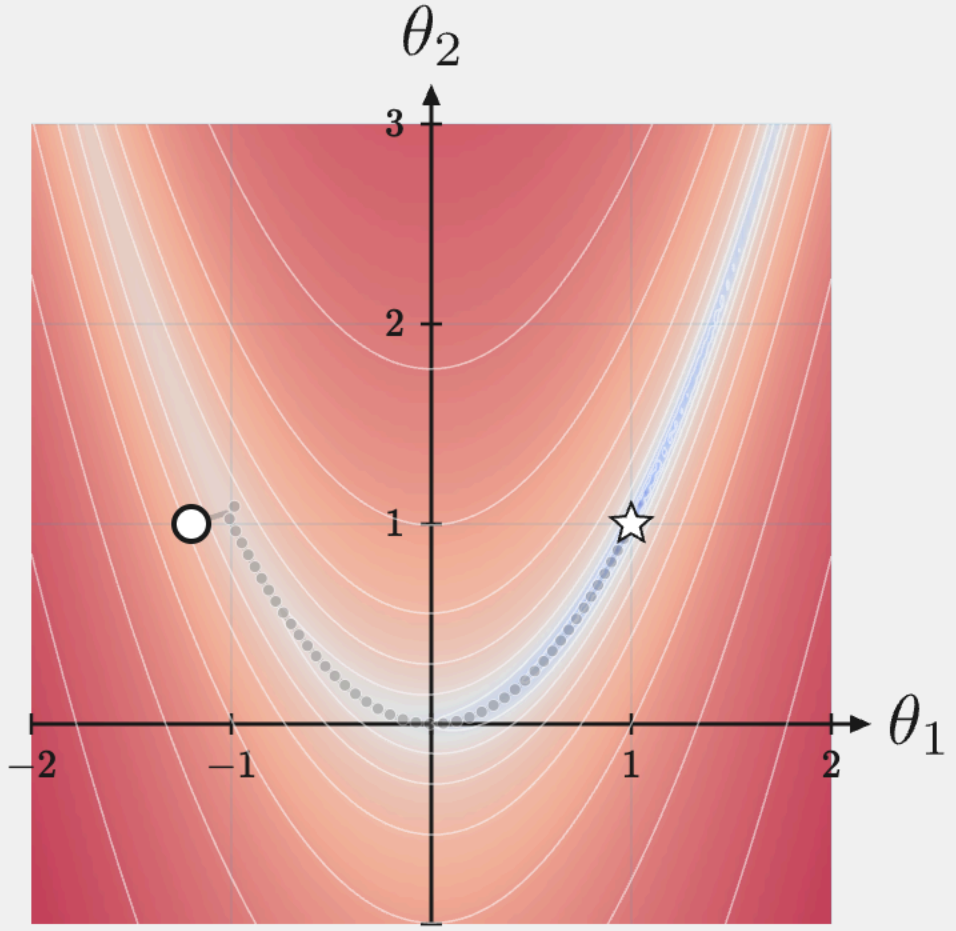
$\alpha = 0.001, \; k = 10000$

$log_{10}\, \alpha = -3.00$



$log_{10}\, k = 4.0$





Feedback



https://forms.gle/JWWzxqNs4ae21iyx7

Please let me know your thoughts and concerns!